DOCUMENTATION
APIs DOCUMENTATION

Presentado por:

Isabella Garces Acosta
Diego Steven Pinzon Yossa
Juan Camilo Rosero Satisteban
Sharick Yelixa Torres Monroy
Laura Sofia Vargas Rodriguez

Profesora:
Liliana Marcela Olarte Mesa

Universidad Nacional de Colombia
Facultad de ingeniería
Departamento de Ingeniería de sistemas e industrial

2025

CONTENT

1. EXTERNAL APIs

a. **Flights:** A Next.js API route handler for the /api/external/places/flights endpoint, designated for server-side flight search operations. Configured with forced dynamic rendering and Node.js runtime, it processes POST requests with comprehensive CORS support via preflight OPTIONS handling. The handler performs extensive validation of request parameters, including: required city names and geographic coordinates with boundary checks (latitude ±90°, longitude ±180°), date fields in ISO 8601 format (YYYY-MM-DD) with temporal logic validation (return date after departure date, no past departures), passenger counts with business rule enforcement (1-9 adults, 0-9 children/infants, infants not exceeding adults), and optional constraints such as cabin class enumeration, maximum stops (0-3), result pagination limits, search radius (1-200 km), and price range boundaries. Upon successful validation, it delegates query execution to the flightsService and returns a standardized GetFlightsResponse object containing flight data, origin/destination airport metadata, and result metadata, with HTTP cache-control directives for 30-minute surrogate caching. Error handling encompasses structured responses for validation failures (400) and internal server errors (500) with detailed logging for debugging.

b. **Places: Hotels:** This hotel search endpoint (/api/external/places/hotels) connects to the Amadeus hotel booking API to provide comprehensive accommodation search functionality. It requires detailed booking parameters including precise check-in/check-out dates, guest configurations, and location coordinates, with optional filters for price ranges, hotel chains, board types, and search radii. The implementation includes sophisticated date validation ensuring chronological correctness and preventing past bookings, along with comprehensive parameter validation for numerical ranges and data formats. The service handles complex hotel search scenarios including multi-room bookings and child occupancy, returning structured hotel data with pricing and availability information, while implementing shorter cache durations (1800 seconds) appropriate for dynamic hotel pricing and availability data.

c. **Places: Restaurants:** This Next.js API route (/api/external/places/restaurants) provides a comprehensive restaurant search service that integrates with Google Places or a similar external API. It accepts detailed search parameters including geographical coordinates, multiple cuisine categories (from "fine_dining" to "fast_food"), price level filters, rating thresholds, and radius limitations. The implementation features robust validation for all input parameters, intelligent filtering of valid restaurant categories, and error handling with informative messages. The route leverages configuration constants for default values and limits, implements CORS support for cross-origin requests, and includes intelligent caching headers (3600-second cache with 7200-second stale-while-revalidate) to

optimize performance while maintaining data freshness for restaurant searches in specific urban areas.

d. **Places: Tourist sites:** This cultural attractions API (/api/external/places/tourist-sites) specializes in discovering points of interest including museums, parks, monuments, and historical sites within a specified geographical area. It accepts location parameters and category filters to return curated cultural destinations with rating information and geographical data. The implementation features streamlined validation focused on the four primary attraction categories, with configurable search radius and minimum rating thresholds. This route serves as a cultural discovery tool for travelers, providing structured information about tourist attractions while implementing long-term caching (3600 seconds) appropriate for relatively stable attraction data, making it efficient for repeated destination research queries.

2. ITINERARIES APIs

a. **Itineraries list & creation API route:** This Next.js API route (/api/itineraries) serves as the primary endpoint for managing travel itinerary collections, providing both retrieval and creation functionalities. The GET method supports sophisticated filtering with query parameters for public/private access, user-specific retrieval, and pagination with skip/limit controls. The POST method includes a unique recursive ID conversion utility that intelligently transforms string IDs within nested day structures into MongoDB ObjectId instances, ensuring proper database relationships. The implementation features comprehensive error handling with detailed logging, CORS configuration for broad client access, and lean document queries for optimized performance. This route forms the backbone of the itinerary management system, enabling users to both browse existing travel plans and create new multi-day itineraries with structured day-by-day activities and pricing information.

b. **Single itinerary CRUD operation API route:** This dynamic route handler (/api/itineraries/[id]) provides complete CRUD (Create, Read, Update, Delete) operations for individual itinerary documents identified by their MongoDB ObjectId. The GET method retrieves a specific itinerary while automatically updating its lastViewedAt timestamp, providing usage analytics. The PUT method allows partial updates with Mongoose validators ensuring data integrity, while the DELETE method provides safe document removal. All operations include extensive validation of the ObjectId format and comprehensive error handling with appropriate HTTP status codes (404 for not found, 400 for invalid IDs, 500 for server errors). This endpoint serves as the detailed management interface for individual travel plans, enabling users to modify, retrieve, or delete specific

itineraries while maintaining data consistency through MongoDB transactions and proper HTTP method semantics.

c. **User-Specific itineraries retrieval API route:** This user-centric API endpoint (/api/itineraries/user/[userId]) specializes in retrieving paginated itinerary collections associated with a specific user account. The implementation leverages Next.js dynamic routing to extract the userId parameter from the URL path, then queries the database with efficient pagination using skip/limit parameters (defaulting to 16 items per page). A notable feature is the use of a custom toItineraryLean transformation function that converts raw MongoDB documents to typed TypeScript objects, ensuring type safety throughout the application. The response includes comprehensive pagination metadata (total count, hasMore flag) enabling infinite scroll or paginated UI implementations. This route is optimized for performance with lean queries and proper CORS headers, serving as the primary interface for users to access their personal collection of saved travel itineraries in a scalable, paginated format.

3. USERS APIs

a. **User profile management API route:** This Next.js API route (/api/users/profile) provides basic user profile management functionality with Firebase authentication integration. The PUT method enables profile updates including name and profile image, utilizing Firebase UID for user identification and performing essential validation for required fields. The GET method retrieves user profiles by Firebase UID query parameter. The implementation connects to MongoDB through a shared database utility, performs basic validation, and returns user documents without additional transformation. This route serves as a straightforward interface for frontend applications to manage basic user profile information, focusing on core update and retrieval operations without complex business logic or extensive validation beyond basic field requirements.

b. **Comprehensive user management API route:** This advanced user management API (/api/users) implements full RESTful CRUD operations with sophisticated validation, pagination, and complex data structures. The GET method supports flexible user queries with pagination (default 50 items per page), search by Firebase UID or email, and structured pagination metadata. The POST method features extensive validation through the validateCreateUserBody function, checking required fields, email format, and complex nested ILocation objects with coordinate validation. The PATCH method allows partial user updates with field-specific validation and prevents modification of immutable Firebase UID. The DELETE method provides user account removal with confirmation responses. The implementation demonstrates professional API design patterns including consistent error response formatting, duplicate detection (with 409 conflict status), MongoDB validation integration, and proper TypeScript typing throughout.

c. **User Onboarding workflow API route:** This specialized onboarding API (/api/users/onboarding) manages the user onboarding process with dual functionality. The GET method checks a user's onboarding completion status by querying the hasCompletedOnboarding flag, enabling conditional UI flows based on user progress. The POST method completes the onboarding workflow, requiring and validating essential user setup data including name and originCity (with comprehensive coordinate and location validation). The implementation prevents duplicate onboarding completions (returning 409 conflict if already completed) and updates multiple user fields simultaneously. This route serves as a focused workflow endpoint that orchestrates the initial user setup experience, separating onboarding logic from general profile management to maintain clean separation of concerns and specialized validation for the critical initial user configuration phase.
d. **Account deletion API route:** This dedicated account deletion endpoint (/api/users/delete) provides a focused interface for permanent user account removal. Unlike the comprehensive user management API's DELETE method, this route accepts the Firebase UID from the request body rather than query parameters, offering a simplified interface for client-side account deletion workflows. The implementation performs basic validation for required Firebase UID, executes the database deletion operation, and returns appropriate success or error messages. This specialized endpoint demonstrates API design principle of creating task-specific routes for critical operations, providing a clear and straightforward interface for the sensitive operation of permanent account removal while maintaining error handling and proper response formatting.